



Draw It Or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	8

Document Revision History

Version	Date	Author	Comments
1.0	05/19/26	Manley Johnson	Initial project design

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The software design *Draw or Lose It* is currently only available on Android but Gaming Room wishes to expand it into a web-based application runs across multiple platforms. The main issue is creating a system that supports multiple teams and players while keeping the information unique and organized.

I propose we design the application using OOP principles. The system should include classes as Game, Team, and Player with each object having a unique identifier. A singleton design pattern should be used to make sure only one instance of the game service exists in memory at one time. This will help prevent duplicate games and allow the application to manage teams, players and game names efficiently.

Requirements

The Gaming Room has requested the following requirements:

- The application must support one or more teams in a game.
- Each team must be able to have multiple players.
- Game and team must be unique so users can check if a name is in use
- Only one instance of the game service should exist in memory at one time.
- Each, game, team, and player should have a unique identifier.
- The game should be web-based so it can support multiple platforms.

Design Constraints

One major design constraint is that the application must work in a web-based distributed environment. This means the game may be accessed by users from different devices, browsers, and operating systems. Because of this, the application must be designed to work consistently across platforms. Another constraint is memory management. Since only one instance of the game service should exist in memory, the singleton design pattern should be used. This prevents the system from creating multiple versions of the same service, which could cause duplicate data or conflicts.

The application also needs to maintain unique names for games and teams. This means the system must check existing names before creating new objects. If this is not handled properly, users could create duplicate games or teams, making the system confusing.

Security and scalability are also important constraints. Since the game will be web-based, the application should be designed so it can support multiple users at the same time while protecting user and game data.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram represents the main objects used in the game application. The primary classes are usually Game, Team, and Player.

The Game class represents a single game session. A game can have one or more teams. This shows a relationship between the Game class and the Team class.

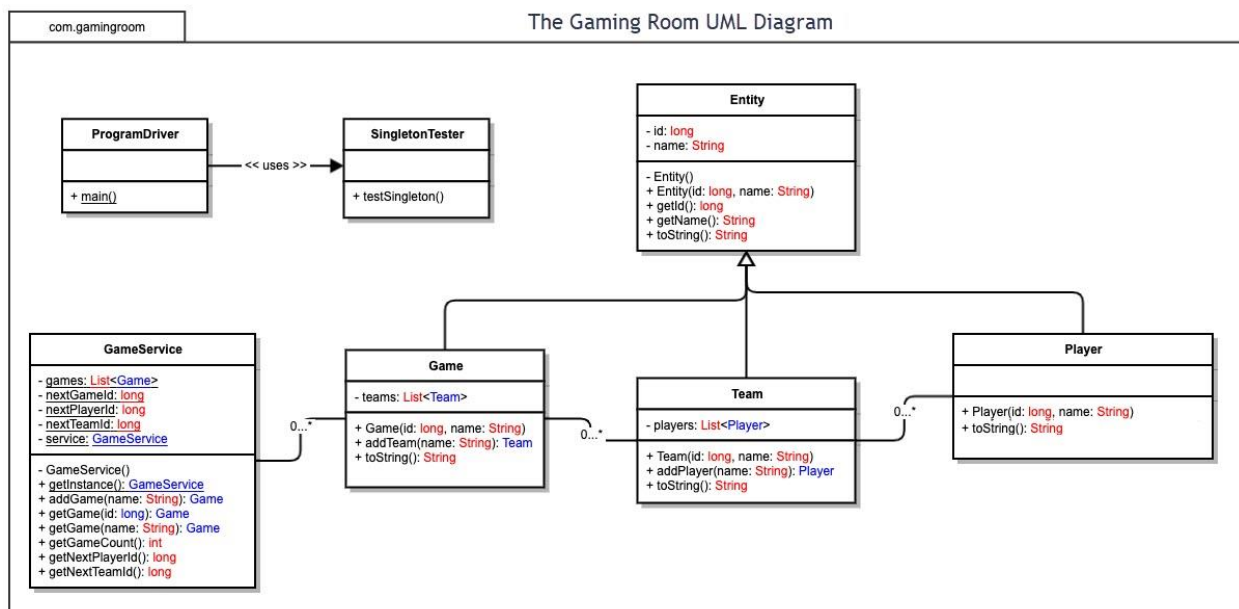
The Team class represents a group of players. Each team can have multiple players assigned to it. This shows a relationship between the Team class and the Player class.

The Player class represents an individual person playing the game. Each player belongs to a team.

The diagram demonstrates **inheritance** if the classes share a common parent class, such as an Entity class that provides common fields like id and name. This helps avoid repeating the same code in multiple classes.

The diagram also demonstrates **encapsulation** because each class contains its own data, such as IDs and names, and controls access to that data through methods.

The singleton pattern demonstrates **abstraction** by hiding the details of how the game service is created and managed. The rest of the program can use the game service without needing to know the full process behind it.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can host a web-based application, but it is not commonly used as a production server environment. While macOS provides strong security and reliability, Apple hardware can be expensive and offers fewer advantages for large scale server deployment. Mac would be more appropriate as a development platform than as the primary server hosting environment.	Linux is likely the strongest option for hosting the Draw It or Lose It web application. It is widely used for web servers because it is stable, secure, scalable, and has low licensing costs. Since The Gaming Room wants to support thousands of players in a distributed environment, Linux can efficiently handle large workloads while remaining cost effective. The primary disadvantage is that administrators may need more technical expertise to configure and maintain Linux servers.	Windows can effectively host a web-based application and may be easier for organizations already familiar with Microsoft products. It provides strong enterprise support and integrates well with other Microsoft technologies. However, Windows Server licensing costs can be higher than Linux, which may increase expenses as the application grows. For The Gaming Room, Windows would be a capable option but may not be the most cost-effective choice.	Mobile devices are not suitable for hosting the server side of the application. Smartphones and tablets do not have the resources necessary to support thousands of users in a distributed environment. Instead, mobile devices should act as clients that connect to a centralized web server.

Client Side	Since the application will be delivered through a web browser, Mac users can access the game without requiring a separate macOS version. Development costs are reduced because the same web application can be used across multiple platforms. Additional development time will be required to test compatibility with browsers such as Safari and Chrome. Developers should have experience with responsive web design and browser testing to ensure a consistent user experience.	Linux users can access the application through standard web browsers such as Firefox and Chrome. Using a browser-based solution reduces development costs because a separate Linux application is not required. Additional testing time may be needed because Linux users can choose from many distributions and browser configurations. Developers should have experience with web technologies and cross browser testing.	Windows has the largest desktop user base, making compatibility testing especially important. A browser-based approach lowers development costs because the same application can support Windows, Mac, and Linux users. Additional development time will be needed to verify compatibility across browsers such as Edge, Chrome, and Firefox. Developers should have expertise in responsive web design and browser compatibility testing.	Supporting Android and iOS devices requires responsive web design that adapts to different screen sizes and touch interfaces. Development costs are reduced compared to building separate native applications, but additional testing time is required across multiple mobile devices and browsers. Developers should have experience designing mobile friendly interfaces and optimizing performance for varying device capabilities.
--------------------	--	---	---	---

Development Tools	Development can be performed using Java, JavaScript, HTML, CSS, and tools such as Visual Studio Code, Eclipse, IntelliJ IDEA, Git, and Xcode. Most development tools are free or offer free versions, although Apple developer services may involve licensing costs. Team members should be familiar with both web development technologies and Mac development environments.	Linux supports development tools such as Visual Studio Code, Eclipse, IntelliJ IDEA, Git, Apache, and Nginx. Many Linux development tools are open source, helping reduce licensing costs. However, development teams may need Linux administration experience to configure servers and deployment environments. Linux expertise can be especially valuable for teams responsible for hosting and maintaining the production server.	Windows supports popular development tools such as Visual Studio Code, Visual Studio, Eclipse, IntelliJ IDEA, Git, Java, JavaScript, HTML, and CSS. Many tools are free, although some Microsoft development products may require paid licenses. Windows is commonly used by developers, which may reduce training requirements and improve productivity. Teams should be familiar with web development and browser compatibility testing.	Mobile support relies primarily on web technologies such as HTML, CSS, JavaScript, and responsive design frameworks. Testing tools may include Android Studio emulators, Xcode simulators, and browser developer tools. While separate Android and iOS development teams are not required for a browser-based application, developers should have experience testing across multiple devices and screen sizes. Apple developer tools may involve licensing costs for iOS testing and deployment.
--------------------------	--	---	---	---

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** The recommended operating platform for The Gaming Room is a Linux-based server environment. Linux is a strong choice because it is stable, secure, cost-effective, and commonly used for hosting web-based applications. Since Draw It or Lose It is currently designed for Android but needs to expand to other operating systems, the best approach is not to build a separate full application for every platform. Instead, The Gaming Room should host the game on a Linux server and allow users to access it through a web-based client. This approach would allow players on Windows, macOS, Linux, Android, and iOS to connect to the same system through a browser or compatible client application. It also supports the client-server model that Draw It or Lose It needs because the server can manage game sessions, users, teams, unique game names, and communication between players.
2. **Operating Systems Architectures:** The recommended platform should use a 64-bit Linux operating system architecture. A 64-bit architecture allows the server to use more memory, handle more processes, and support a larger number of connected users than a 32-bit system. Linux also uses a layered architecture where the kernel manages hardware resources, memory, processes, file systems, and network communication. For Draw It or Lose It, this architecture is useful because the game needs reliable server-side control. The server must keep track of active games, teams, players, and unique identifiers. Linux supports multitasking and process isolation, which helps prevent one failed process from affecting the entire system. This makes it a strong choice for a multiplayer game environment where multiple users may be connected at the same time.
3. **Storage Management:** The Gaming Room should use a relational database management system, such as MySQL or PostgreSQL, along with the Linux file system. The database should store structured game data such as user accounts, player profiles, teams, game names, game sessions, scores, and other records that need to be organized and searched efficiently. The Linux file system can be used to manage application files, logs, configuration files, and any stored image or drawing-related data. File system components such as directories, files, metadata, and permissions help organize and protect the data. Using a database together with the server file system gives The Gaming Room a reliable way to store both structured game information and supporting application files.
4. **Memory Management:** Linux uses memory management techniques such as virtual memory, paging, process isolation, and memory allocation to help applications run efficiently. Each running process receives its own protected memory space, which helps prevent one process from interfering with another. This is important for Draw It or Lose It because many users may be connected to the game at the same time. Since the application is Java-based, memory management will also be supported by the Java Virtual Machine. The JVM manages heap memory and uses garbage collection to remove objects that are no longer needed. This helps reduce memory leaks and keeps the application more

stable. The server should also be designed so that only necessary objects remain in memory, especially because the game design already requires only one instance of the game service through the singleton pattern.

5. **Distributed Systems and Networks:** Draw It or Lose It should use a distributed client-server system. In this design, the Linux server hosts the main application logic, database access, authentication, and game session management. The client devices connect to the server over the internet using standard network protocols.

This allows players using different devices and operating systems to participate in the same game without needing the full game logic installed locally. The clients would send requests to the server, and the server would respond with updated game information. A REST API can be used to support communication between the client and server. HTTPS should be used to protect data while it travels across the network.

This distributed design also makes the system easier to maintain. Updates can be made on the server side instead of requiring every user to install a new version of the application.

6. **Security:** Security is important because The Gaming Room will need to protect user accounts, game data, and communication between different platforms. The system should use HTTPS to encrypt data sent between the client and server. User passwords should not be stored as plain text; they should be hashed and salted before being saved in the database.

The application should also use authentication and authorization. Authentication confirms that a user is who they claim to be, while authorization controls what that user is allowed to access. For example, normal players should only access player-level features, while administrators should have access to management features.

Linux also provides security advantages through file permissions, user roles, process isolation, and firewall configuration. These features can help protect server files, limit unauthorized access, and reduce the risk of one part of the system affecting another. Regular updates, backups, and access control should also be used to protect the system over time.